4 První aplikace v PHP, práce s proměnnými a konstantami, operátory

V této kapitole se dozvíte:

- Jakým způsobem pracovat s proměnnými a konstantami v PHP.
- Jaké datové typy existují v PHP.
- Jaké operátory je možné použít v PHP.

Po jejím prostudování byste měli být schopni:

- Umět vypsat text na obrazovku (do okna prohlížeče).
- Umět pracovat s proměnnými v PHP.
- Znát operátory dostupné v PHP.

Klíčová slova této kapitoly:

Proměnná, konstanta, datový typ, echo, operátory.

Doba potřebná ke studiu: 6 hodin

Průvodce studiem



Studium této kapitoly představuje úvod do tvorby aplikací v PHP. Pro výpis textu na obrazovku (do okna prohlížeče) slouží příkaz echo. Seznámíte se s různými způsoby použití tohoto příkazu. Dále se seznámíte s dostupnými datovými typy v PHP, naučíte se pracovat s proměnnými a konstantami. V neposlední řadě se seznámíte s operátory, které je možné použít v PHP.

Jazyk HTML používaný pro tvorbu webových stránek umožňuje vytvářet pouze statické webové stránky. Kód stránky v tomto jazyce je uložen v souboru s příponou .htm nebo .html a při požadavku klienta na daný soubor je mu odeslán obsah tohoto souboru. WWW prohlížeč na straně klienta se pak postará o to, aby se obsah souboru zobrazil správně, tj. aby se texty naformátovaly podle použitých kaskádových stylů, aby byl funkční kód v jazyce JavaScript.

Skripty v jazyce PHP jsou uloženy jako soubory s příponou .php a samotný kód PHP je čistý text ve formátu ASCII. Kód PHP tedy můžeme vytvářet v libovolném textovém editoru (notepad v MS Windows, PSPad atd.). Soubor s příponou PHP může obsahovat i HTML kód. HTML kód se odešle klientovi ve stejné podobě v jaké je uveden v PHP souboru. Pokud je potřeba vložit kód PHP, je nutné jej označit pomocí speciálních značek. K tomu se v zásadě používají dvě značky, obě jsou párové. První značka začíná `<?php` a končí `?>`, druhá začíná `<?` a končí taktéž `?>`. Použití druhé značky nemusí vždy spolehlivě fungovat, proto je lepší používat raději první značku. Také proto bude v dalším textu používána tato značka.

Výpis 4.1 Ukázka použití značky pro označení kódu PHP.

```
<?php
  …kód php…
?>
```

Struktura kódu PHP je velmi podobná struktuře kódu v Javě nebo v C/C++. Každý příkaz musí končit středníkem `;`. Výjimkou je případ, kdy soubor PHP obsahuje pouze jeden příkaz, v tom případě se jeho vykonání provede i bez uvedení středníku za tímto příkazem. Kromě klíčových slov, identifikátorů proměnných, konstant nebo funkcí můžeme v kódu používat i komentáře. Při zpracování PHP skriptu je komentář přeskočen. Používání komentářů bývá často programátory opomíjeno, ale při větších projektech je vhodné používat komentáře, protože údržba velkého projektu bývá náročná. Navíc programátoři se mohou měnit a nově příchozí programátor má ze začátku problém se v kódu svého předchůdce vůbec zorientovat. PHP nabízí použití dvou typů komentářů – jednořádkový nebo víceřádkový komentář. Syntaxe pro použití obou typů komentářů je stejná jako v jazyce Java.

Výpis 4.2 Ukázka použití komentáře v PHP.

```
<?php
  // jednořádkový komentář
  // další jednořádkový komentář

  /*
  toto je
  víceřádkový
  komentář
  */
?>
```

Jednořádkový komentář začíná za znaky `//` a končí na konci aktuálního řádku. Víceřádkový komentář je tvořen všemi řádky textu, které jsou uvedeny mezi `/*` a `*/`. Kromě doplnění dodatečných informací ke kódu PHP je možné využít komentáře i k ladění programů. Postupným „zakomentováním“ jednotlivých částí kódu můžeme odhalit, která část způsobuje chybu.

4.1 První PHP skript

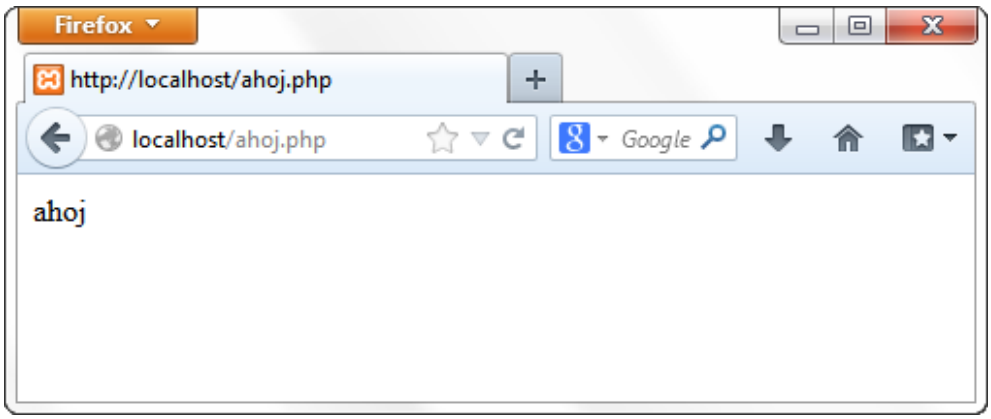
Ve většině programovacích jazyků bývá zvykem, že první vytvořený program vypíše na obrazovku pozdrav. Ani v tomto případě to nebude jinak, náš první PHP skript vypíše slovo „ahoj“ do okna prohlížeče. Následující kód je uložen v souboru `ahoj.php`, který je uložen v kořenovém adresáři serveru (v tomto

případě byl použit server Apache). Pro všechny následující příklady v této opoře bude použit webový server Apache nainstalovaný lokálně na počítači. Adresa tohoto serveru je `localhost`, všechny zobrazené PHP skripty tedy budou umístěny na adrese `http://localhost`.

Výpis 4.3 Obsah souboru ahoj.php

```
<?php
  echo "ahoj";
?>
```

Následující obrázek zobrazuje výsledek vykonání uvedeného PHP kódu.



Obrázek 4.1 Zobrazení souboru ahoj.php v prohlížeči.

Jak vidíme ve Výpisu 4.3, kromě značek pro označení začátku a konce PHP kódu, je zde uveden pouze jeden příkaz. Příkaz `echo` vypíše text do výstupu, který je odeslán ze serveru klientovi. V tomto případě je vypsán řetězec znaků uvedený v uvozovkách. Pro vytvoření řetězcové konstanty je v PHP možné použít jak klasické uvozovky (viz. Výpis 4.3) nebo apostrofy – jednoduché uvozovky. Příkaz



```
echo 'ahoj';
```

by tedy měl úplně stejný efekt. Kromě toho je možné u příkazu echo použít ještě závorku. Výpis by tedy mohl vypadat i následovně:

```
echo "ahoj"; nebo echo 'ahoj';
```

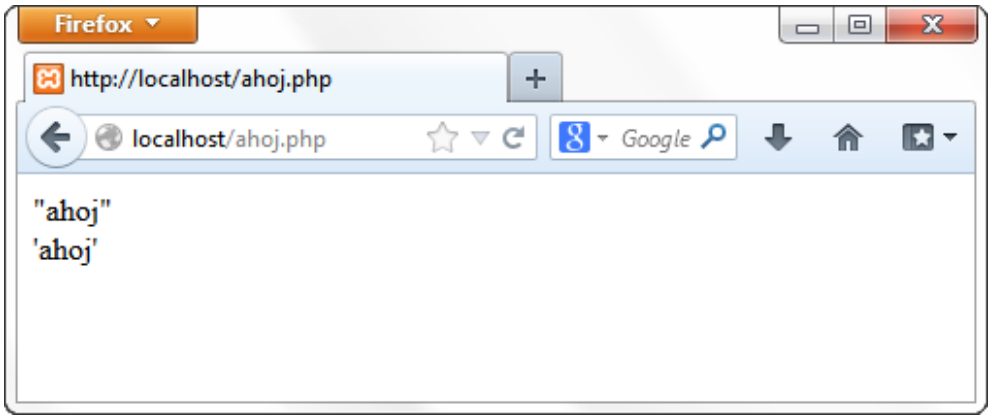
Pokud bychom v rámci vypisovaného řetězce chtěli vypsat uvozovky a pro ohraničení vypisovaného řetězce použijeme také uvozovky, je nutné použít pro vypsání uvozovek escape sekvenci – před uvozovky umístit obrácené lomítko `\`. Totéž platí v případě použití apostrofů, tj. jednoduchých uvozovek.

Následující výpis ukazuje použití escape sekvence a na obrázku je pak zobrazen výsledný výstup v prohlížeči. Protože standardně je generovaným výstupem HTML kód, pro odřádkování je nutné použít značku `<br>`.

Výpis 4.4 Použití escape sekvence.



```
<?php
  echo "\"ahoj\"<br>";
  echo '\'ahoj\'';
?>
```



Obrázek 4.2 Ukázka použití escape sekvence v prohlížeči.

Kromě uvedených escape sekvencí existují ještě další escape sekvence, které jsou známé i z jiných programovacích jazyků. Escape sekvence `\n` představuje přechod na nový řádek. Pokud tuto escape sekvenci použijeme v předchozím výpisu, tj. jeho tvar bude:

```
<?php
  echo "\"ahoj\"<br>\n";
  echo '\'ahoj\'';
?>
```

v okně prohlížeče neuvidíme žádný rozdíl. Rozdíl ale bude patrný, pokud si zobrazíme zdrojový kód stránky. Bez použití escape sekvence `\n` je ve zdrojovém kódu stránky text

```
"ahoj"<br>'ahoj'
```

Pokud escape sekvenci použijeme, zdrojový kód stránky bude

```
"ahoj"<br>

'ahoj'
```

Další escape sekvencí je tabulátor, který vytvoříme pomocí `\t`. Escape sekvence `\r` představuje návrat vozíku na začátek řádku. Pokud potřebujeme do výstupu vypsat obrácené lomítko, je nutné použít escape sekvenci `\\`. A pokud potřebujeme vypsat znak dolaru, je k tomuto účelu nutné použít escape sekvenci `\$`. Znak dolaru má totiž v PHP speciální význam.

K tomu, abychom do výstupu vypsali uvozovky nebo apostrofy, můžeme použít i jiný postup. Stačí zkombinovat použití uvozovek a apostrofů ve výstupním řetězci. Vnější znaky vždy ohraničují výstupní řetězec, vnitřní znaky lišící se od vnějších znaků se vypíší tak, jak jsou. Takže pomocí následujícího kódu vypíšeme text s uvozovkami nebo apostrofy i bez použití escape sekvence.

Výpis 4.5 Výpis uvozovek nebo apostrofů bez escape sekvence.

```
<?php
  echo '"ahoj"<br>';
  echo "'ahoj'";
?>
```



Výstup tohoto PHP kódu bude stejný jako na obrázku 4.2. Ostatní escape sekvence (přechod na nový řádek, tabulátor atd.) fungují ale pouze pokud jsou uvedeny v klasických uvozovkách. Pokud je uvedeme v apostrofech, zobrazí se ve výstupu v prohlížeči a nemají na zdrojový kód stránky žádný vliv.

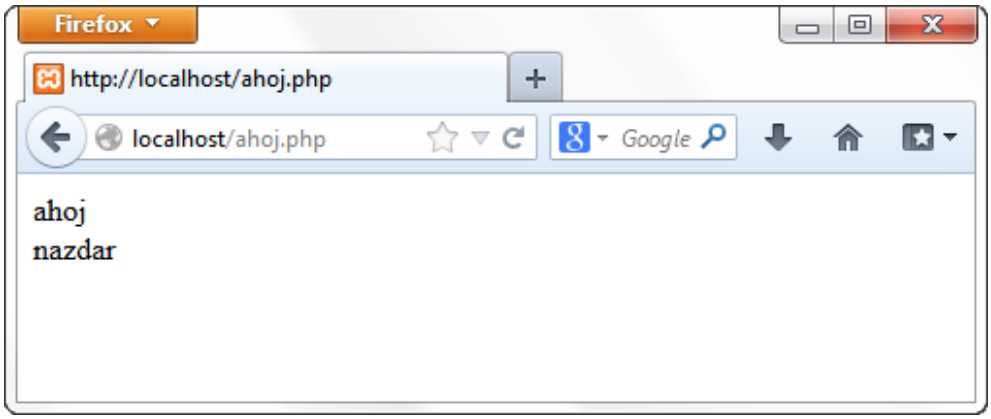
Možnost použití kombinace apostrofů a klasických uvozovek je výhodná zejména při generování HTML kódu do výstupu. HTML kód totiž zpravidla obsahuje i parametry jednotlivých značek (např. pro značku INPUT je to parametr type). Hodnoty parametrů se zpravidla uvádějí v uvozovkách. Pokud bychom pro označení začátku a konce výstupu použili klasické uvozovky, musíme před všechny uvozovky ve výstupním HTML kódu uvést znak obráceného lomítka. Pokud ovšem na začátku a na konci HTML kódu použijeme apostrof, nemusím se tímto zdržovat. HTML kód je navíc bez escape sekvencí u každých uvozovek o mnoho přehlednější.

Pokud je potřeba spojit více řetězců dohromady nebo spojit řetězce spolu s hodnotami proměnných, je možné použít operátor `.` (tečka), který spojení provede.

Výpis 4.6 Ukázka spojení řetězců.

```
<?php
  echo "ahoj" . "<br>" . "nazdar";
```

```
?>
```



Obrázek 4.3 Výsledek spojení řetězců zobrazený v prohlížeči.

4.2 Proměnné a datové typy



Jazyk PHP umožňuje používání proměnných, stejně jako je tomu v jiných programovacích jazycích. Proměnná má svůj identifikátor (název) a hodnotu. Identifikátor proměnné vždy začíná znakem `$` (dolar). Proto je nutné použít escape sekvenci, pokud chceme vypsat znak dolaru. Pro zbytek identifikátoru (za znakem dolaru) platí, může obsahovat písmena anglické abecedy, čísla a podtržítko. Začínat může písmenem nebo podtržítkem. Identifikátory proměnných jsou case senzitivní, tj. rozlišují se v nich malá a velká písmena. Zpravidla se identifikátory píší malými písmeny. Následují ukázky identifikátorů.

Výpis 4.7 Ukázka identifikátorů.

```
<?php
  $cislo;
  $_promenna;
  $hodnota123;
  $Hodnota123; /* nová proměnná, identifikátory
   $hodnota123 a $Hodnota123 se liší */
  $3abc; // chybný identifikátor - začíná číslem
?>
```

Pro hodnoty proměnných jsou v PHP různé datové typy, celkem existuje 6 hlavních datových typů – logická hodnota, celé číslo, desetinné číslo, řetězec znaků, pole a objekt.

4.2.1 Jednoduché datové typy

Logická hodnota

Logický typ se v PHP nazývá Boolean. Slouží k uchování hodnoty „pravda“ nebo „nepravda“. Hodnoty tohoto typu jsou samozřejmě v angličtině, tj. true a false. Hodnoty nejsou case sensitivní, takže můžeme napsat i TRue nebo True.

Celé číslo

Název celočíselného typu je Integer. Slouží k uchování celých čísel. Čísla mohou být kladná nebo záporná. Rozsah tohoto typu se liší v závislosti na použité platformě, ale obvykle bývá zhruba od -2 miliard do +2 miliard. Hodnota může být zadána v desítkové, osmičkové, šestnáctkové nebo binární soustavě.

Desetinné číslo

Datový typ pro uložení reálné hodnoty se v PHP nazývá Float nebo Real. Rozsah a přesnost typu je závislá na platformě, ale obvykle platí, že přesnost tohoto typu je 14 desetinných míst, rozsah je velmi velký, zhruba 1.8e308. Při zadávání hodnoty je nutné použít desetinnou tečku, je možné použít i matematický zadání pomocí exponentu.

Řetězec znaků

Tento datový typ se nazývá String a slouží k uchování posloupnosti znaků. Jeho délka je v PHP prakticky neomezená. K vytvoření hodnoty typu String je možné použít buď klasické uvozovky nebo apostrofy (jednoduché uvozovky).

Na rozdíl od jiných programovacích jazyků, v PHP neexistuje deklarace proměnné. Datový typ proměnné je určen v okamžiku uložení hodnoty do proměnné. Přiřazení hodnoty do proměnné je realizováno pomocí znaku = (rovná se). Pokud tedy do proměnné přiřadíme hodnotu `10`, automaticky se její typ nastaví na celé číslo. Pro výpis hodnoty proměnné se používá příkaz `echo`, např. `echo $promenna;`. Název proměnné může být uveden i v rámci vypisovaného řetězce. Následují ukázky vytvoření proměnných jednotlivých datových typů.

Výpis 4.8 Ukázka vytvoření logické proměnné.

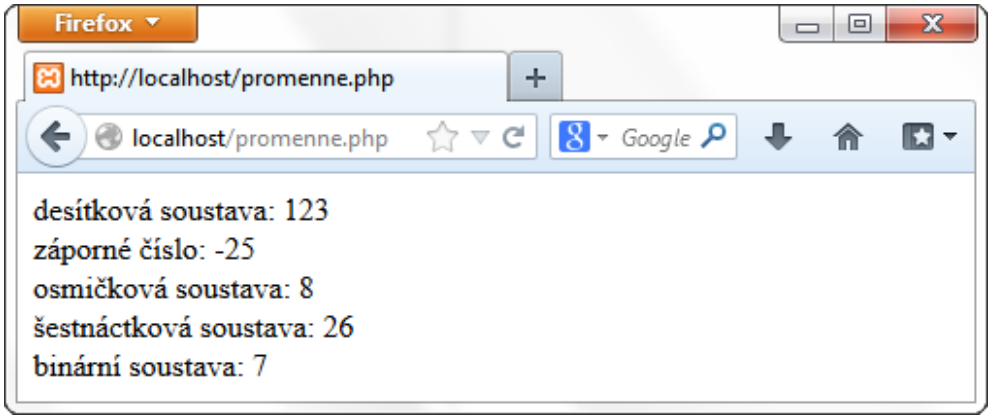
```
<?php
  // logická proměnná
  $logicka1 = true;
  $logicka2 = false;
?>
```

Výpis 4.9 Ukázka vytvoření celočíselné proměnné.



```
<?php
  // celé číslo
  // desítková soustava
  $cislo = 123;
  echo "desítková soustava: " . $cislo . "<br>";
  // záporné číslo
  $cislo = -25;
  echo "záporné číslo: $cislo<br>";
  // osmičková soustava
  $cislo = 010;
  echo "osmičková soustava: " . $cislo . "<br>";
  // šestnáctková soustava
  $cislo = 0x1A;
  echo "šestnáctková soustava: " . $cislo . "<br>";
  // binární soustava
  $cislo = 0b111;
  echo "binární soustava: " . $cislo . "<br>";
?>
```

Výsledkem v prohlížeči bude výstup zobrazený na následujícím obrázku.



Obrázek 4.4 Výsledek zobrazený v prohlížeči.

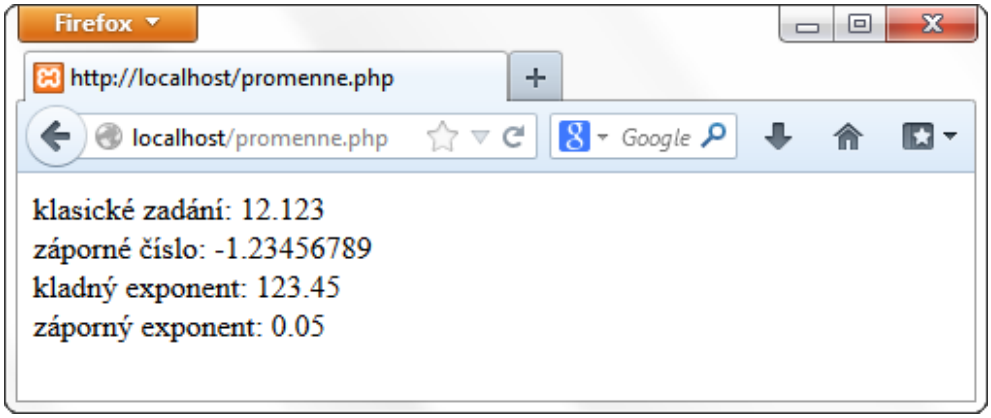
Výpis 4.10 Ukázka použití reálných proměnných.



```
<?php
  // realné číslo
  // klasické zadání
  $cislo = 12.123;
  echo "klasické zadání: " . $cislo . "<br>";
  // záporné číslo
  $cislo = -1.23456789;
  echo "záporné číslo: " . $cislo . "<br>";
  // zadání s kladným exponentem
  $cislo = 1.2345e2;
```

```
  echo "kladný exponent: " . $cislo . "<br>";
  // zadání se záporným exponentem
  $cislo = 5e-2;
  echo "záporný exponent: " . $cislo . "<br>";
?>
```

Výsledek použití reálné proměnné zobrazený v prohlížeči je zobrazen na následujícím obrázku.



Obrázek 4.5 Výsledek zobrazený v prohlížeči.

Uvedené datové typy patří mezi jednoduché typy. Složitějším datovým typem je pole.

4.2.2 Složitější datové typy

Pole

Datový typ pole představuje strukturu, v PHP je pole v podstatě mapou, ve které jsou jednotlivé prvky. Každý prvek má svůj klíč a svou hodnotu. Klíč můžeme nazývat také jako index. Klíčem (indexem) prvku může být celé číslo nebo řetězec. Pokud je klíčem řetězec hovoříme o tzv. asociativním poli. Pole v PHP nemusí být homogenní, tj. každý prvek pole může mít jiný datový typ. Vytvoření pole je v PHP možné několika způsoby. První způsob je vyjmenování prvků, které do pole patří.

Výpis 4.11 Vytvoření pole vyjmenováním prvků.

```
<?php
  // klíč není uveden
  $cisla[] = 10;
  $cisla[] = 20;
  $cisla[] = 30;
  $cisla[] = 40;
```



```
  // klíčem v poli je číslo, ale čísla netvoří řadu
  $teploty[0] = 10.7;
  $teploty[4] = 10.3;
  $teploty[8] = 12.8;
  $teploty[12] = 16.8;

  // klíčem v poli je řetězec
  $jmena["prvni"] = "Petr";
  $jmena["druhe"] = "Jana";
  $jmena["treti"] = "Martin";
  $jmena["ctvrte"] = "Pavla";
?>
```

Při vyjmenování prvků pole `$cisla` nejsou uvedeny indexy (klíče). Prvkům jsou tedy indexy přiřazeny automaticky, jejich číslování začíná od nuly. První prvek pole bude mít index `0`, druhý index `1` atd. PHP umožňuje, aby číselné indexy netvořily souvislou řadu čísel. Ukázka vytvoření takového pole je pole `$teploty`. Indexy prvků by mohly představovat například hodinu, kdy byla daná teplota naměřena. Poslední pole, `$jmena`, obsahuje jako klíče řetězce, v tomto případě už hovoříme o asociativním poli.

Druhou možností, jak vytvořit pole je použití funkce `array()`. Této funkci předáme jako parametry hodnoty prvků v poli. Indexy prvků budou tvořeny čísly, první prvek bude mít index `0`. Pokud chceme změnit indexy prvků, je potřeba použít operátor `=>`. Následující výpis zobrazuje ukázku vytvoření stejných polí jako v předchozím výpisu.

Výpis 4.12 Ukázka vytvoření pole pomocí funkce array().



```
<?php
  // klíč není uveden
  $cisla = array(10, 20, 30, 40);

  // klíčem v poli je číslo, ale čísla netvoří řadu
  $teploty = array(0 => 10.7, 4 => 10.3, 8 => 12.8,
   12 => 16.8);

  // klíčem v poli je řetězec
  $jmena = array("prvni" => "Petr", "druhe" =>
   "Jana", "treti" => "Martin", "ctvrte" => "Pavla");
?>
```

Přístup k prvkům pole je stejný jako v jiných programovacích jazycích, uvedeme název pole a v hranatých závorkách index požadovaného prvku.

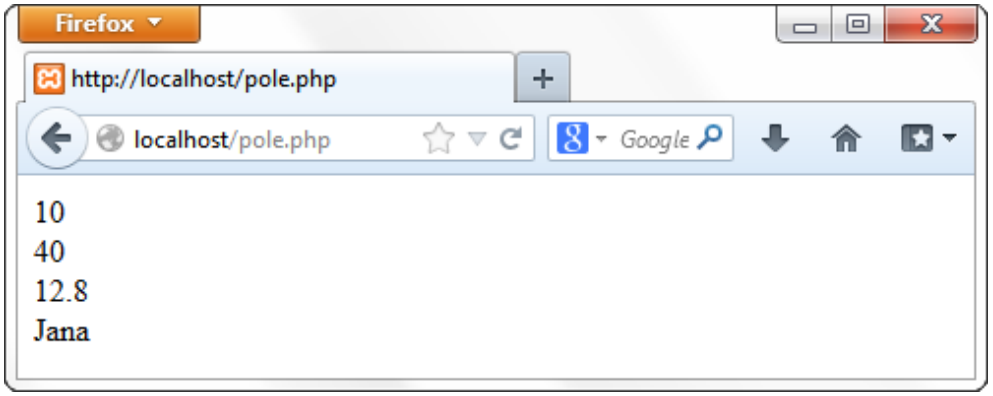
Následující výpis předpokládá, že jsou existují pole vytvořená v předchozím výpisu.

Výpis 4.13 Ukázka výpisu prvků pole.

```
<?php
  echo $cisla[0] . "<br>";
  echo $cisla[3] . "<br>";
  echo $teploty[8] . "<br>";
  echo $jmena["druhe"] . "<br>";
?>
```



Výstup zobrazený v prohlížeči je na následujícím obrázku.



Obrázek 4.6 Zobrazení vypsaných prvků pole v prohlížeči.

Ke zjištění počtu prvků v poli slouží funkce `count()`, která má jako parametr název pole, u kterého chceme počet prvků zjistit. K procházení polí je možné použít cykly, stejně jako v případě jiných programovacích jazyků. Procházení pole, jehož prvky jsou indexovány spojitě, je velmi jednoduché a příliš se neliší procházení pole v Javě. V případě nespojitě indexovaného pole je postup trošku odlišný. Oba postupy si uvedeme v části věnované cyklům.

Jazyk PHP umožňuje používat i vícerozměrná pole. Kromě klasického jednorozměrného pole bývá také občas používáno dvojrozměrné pole, které vznikne tak, že prvky pole nejsou jednoduchého typu, ale jsou to také pole. Tímto způsobem je možné vytvářet i vícerozměrná pole, ale jejich praktické využití není příliš velké. Dvojrozměrné pole bývá používáno například pro reprezentaci matic. Pro procházení dvojrozměrným polem je potřeba použít dva vnořené cykly, kdy vnější cyklus prochází prvky vnějšího pole a v každém kroku se pak pomocí vnitřního cyklu projdou prvky vnitřního pole.

Objekt

Datový typ objekt souvisí s objektově orientovaným programováním, které bude podrobněji probráno v následujících kapitolách.

4.2.3 Přetypování proměnných

Datový typ proměnných v PHP lze změnit, a to pomocí automatického nebo explicitního přetypování. Jak již bylo uvedeno, datový typ proměnné se určí přiřazením hodnoty do proměnné. Automatické přetypování se provede v případě, kdy proměnné přiřadíme hodnotu jiného typu.

Výpis 4.14 Automatické přetypování proměnné.



```
<?php
  $a = 10;
  $a = 2.5;
  $a = "ahoj";
?>
```

Prvním příkazem do proměnné `$a` přiřadíme hodnotu `10` a typ proměnné se nastaví na celé číslo (integer). Hodnota proměnné `$a` je `10`. Druhým příkazem změníme hodnotu proměnné `$a` na `2.5`, ale současně se automaticky změní i datový typ této proměnné – novým datovým typem je desetinné číslo (double). Poslední příkaz opět změní hodnotu proměnné `$a` na řetězec „`ahoj`“ a současně se změní i datový typ na řetězec (string).

Automatická konverze funguje i v případě převodu řetězce na číslo. Pokud řetězec začíná číslicemi, za nimiž následují jiné znaky, převedou se číslice ze začátku řetězce na číslo a zbytek řetězce je ignorován. Výsledkem převodu je datový typ integer. Například řetězec „`10 lidí`“ bude převeden na číslo `10`. Pokud je celý řetězec tvořen znaky odpovídajícími desetinnému číslu, je tento řetězec převeden na typ double. Řetězec „`3.5`“ bude převeden na číslo `3.5`, řetězec „`1.5231e2`“ na číslo `152.31`. Pokud řetězec obsahuje jiné znaky, které nemohou být převedeny na desetinné číslo, je převedena pouze část před desetinnou čárkou a výsledkem je celé číslo. Řetězec „`12.5 stupně Celsia`“ bude převeden na číslo `12`. Pokud řetězec začíná znakem − (mínus), je výsledkem převodu záporné číslo (platí v případě celých i desetinných čísel.

Další automatickou konverzí je přetypování mezi dvěma numerickými typy. Pokud je provedena numerická operace mezi celým a desetinným číslem, výsledkem je desetinné číslo.

Explicitní přetypování provedeme uvedením nového datového typu před názvem proměnné nebo před hodnotou, kterou chceme přetypovat. Pro převod na celočíselný typ se používá klíčové slovo `integer` nebo `int`. Převodem desetinného čísla na celé číslo se převede pouze celá část původní hodnoty, desetinná část je ignorována. Převodem desetinného čísla `10.9` na celé číslo vznikne celé číslo `10`. Pro převod na desetinné číslo se používají klíčová slova `double`, `float` nebo `real`. Převodem celého čísla na desetinné se k původní hodnotě přidá desetinná část. Výsledkem převodu celého čísla `5` na desetinné číslo bude desetinné číslo `5.0`.

4.2.4 Funkce pro práci s proměnnými



Jazyk PHP obsahuje několik vestavěných funkcí pro práci s proměnnými. První funkcí je `gettype()`, která vrací datový typ proměnné. Parametrem funkce je název proměnné, u které chceme datový typ zjistit. Vrácená hodnota představuje název datového typu. Hodnotami, které může funkce vrátit, jsou „`integer`" (celé číslo), „`double`" (desetinné číslo), „`string`" (řetězec), „`array`" (pole), „`object`" (objekt), „`class`" (třída) a „`unknown type`" (neznámý typ).

Druhou funkcí je funkce `settype()`, která převede proměnnou na požadovaný datový typ. Funkce má dva parametry. Prvním parametrem jen název proměnné a druhým datový typ, který chceme proměnné nastavit. Převod lze provést na datové typy `array`, `double`, `integer`, `object` a `string`. Pokud nelze požadovaný typ nastavit, vrátí funkce logickou konstantu `false`.

Funkce `isset()` se používá pro zjištění, zda byla proměnné přiřazena hodnota. Parametrem funkce je název proměnné. Funkce vrací logickou konstantu `true` (hodnota byla proměnné přiřazena) nebo `false` (hodnota nebyla proměnné přiřazena). Funkce `unset()` se používá k odstranění dané proměnné a uvolnění paměti, kterou proměnná zabírá. Parametrem funkce je opět název proměnné.

Funkce `empty()` je opakem funkce `isset()`. Pokud proměnné nebyla přiřazena hodnota, má nulovou hodnotu nebo hodnotu prázdného řetězce, funkce vrací `true`. V opačném případě vrací `false`.

Pro otestování, zda je proměnná konkrétního typu slouží funkce `is_typ()`. Místo slova `typ` je v názvu funkce uveden konkrétní datový typ. Pro

otestování, zda proměnná představuje celé číslo slouží funkce `is_int()`, `is_integer()` nebo `is_long()`. Pro otestování, zda se jedná o desetinné číslo jsou to funkce `is_double()`, `is_float()` a `is_real()`. Pro zbylé datové typy jsou to funkce `is_string()` (řetězce), `is_array()` (pole), `is_object()` (objekt). Všechny funkce mají jako parametr název proměnné a vracejí `true`, pokud je proměnná daného typu nebo `false`, pokud je jiného typu.

Výpis 4.15 Ukázka použití funkcí pro práci s proměnnými.



```
<?php
  $a = 20;
  $b = -2.5;
  $c = array("a", "b", "c");
  $d = "auto";
  echo gettype($a) . "<br>"; // vypíše integer
  settype($a, "double");
  echo gettype($a) . "<br>"; // vypíše double
  echo gettype($b) . "<br>"; // vypíše double
  echo gettype($c) . "<br>"; // vypíše array
  echo gettype($d) . "<br>"; // vypíše string
  echo empty($e) . "<br>"; // vypíše 1
  echo is_double($b) . "<br>"; // vypíše 1
  unset($b);
  echo isset($b) . "<br>"; // vrací false, nevypíše
   nic
?>
```

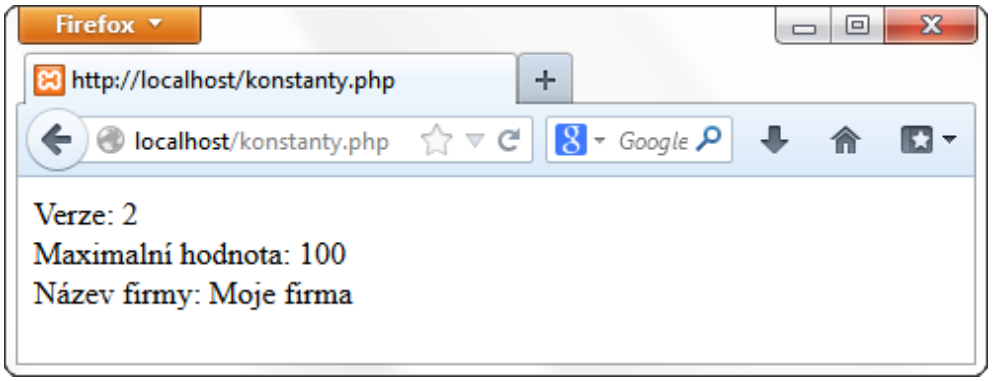
4.3 Konstanty



Na rozdíl od proměnných, jejichž hodnota se může během vykonávání skriptu měnit, konstanty mají po prvním přiřazení stále stejnou hodnotu. Názvy (identifikátory) konstant nezačínají znakem dolaru `$`. Konvencí pro názvy konstant je, že se píši pouze velkými písmeny, případně se používá znak podtržítka `_`. Pro vytvoření konstanty se v PHP používá funkce `define()`, která má dva parametry. Prvním parametrem je název konstanty, který musí být uveden v uvozovkách. Druhým parametrem je hodnota. Číselné hodnoty nemusí být uvedeny v uvozovkách. Pro otestování, zda nějaká konstanta existuje, slouží funkce `defined()`, která má jako parametr název konstanty. Pokud konstanta existuje, funkce vrátí `1`, pokud neexistuje, vrátí `0`. Pokud chceme v kódu pracovat s hodnotou konstanty, stačí uvést její název.

Výpis 4.16 Ukázka definování a vypsání konstant.

```
<?php
  define("VERZE", 2);
  define("MAX", 100);
  define("FIRMA", "Moje firma");
  echo "Verze: " . VERZE . "<br>";
  echo "Maximální hodnota: " . MAX . "<br>";
  echo "Název firmy: " . FIRMA . "<br>";
?>
```



Obrázek 4.7 Výpis konstant v prohlížeči.

4.4 Operátory a výrazy

V jazyce PHP je možné použít unární nebo binární operátory a nebo ternární operátor.

4.4.1 Unární operátory

Prvním unárním operátorem je znaménko mínus `-`, pomocí které je možné vytvořit opačné číslo k nějakému číslu (tj. číslo, které má opačné znaménko než původní hodnota. Dalšími dvěma unárními operátory jsou operátory inkrementace `++` a dekrementace `--`, které slouží ke zvýšení (inkrementace) nebo snížení (dekrementace) hodnoty proměnné o `1`. Oba operátory mohou být umístěny jak před, tak za proměnnou, ke které se vztahují. Na hodnotu této proměnné nemá umístění operátoru vliv. Hodnota této proměnné bude v obou případech zvýšena nebo snížena o `1`. Rozdíl nastane, pokud je inkrementovaná nebo dekrementovaná proměnná součástí jiného výrazu. Pokud je operátor umístěn před proměnnou, je nejprve provedena inkrementace nebo dekrementace a poté se vyhodnotí zbytek výrazu. Pokud je operátor umístěn za proměnnou, je nejprve vyhodnocen výraz s původní hodnotou proměnné a poté je provedena inkrementace nebo dekrementace.

Výpis 4.17 Ukázka použití unárních operátorů.



```
<?php
  $a = 10;
  $b = -$a; // $b = -10

  $c = -5;
  $d = -$c; // $d = 5

  $e = 10;
  $f = ++$e; // $f = 11, $e = 11
  $e = 10;
  $g = $e++; // $g = 10, $e = 11
  $e = 10;
  $h = 2 * ++$e; // $h = 22, $e = 11
  $e = 10;
  $i = 2 * $e++; // $i = 20, $e = 11
?>
```

4.4.2 Binární operátory

Jako operátor přiřazení je v PHP používán znak = (rovná se). Pomocí tohoto operátoru můžeme přiřadit hodnoty proměnným. S tímto operátorem jsme se již setkali v předchozích kapitolách.

Binární operátory jsou operátory, které jsou spojené se dvěma operandy. V PHP existují tři skupiny binárních operátorů. Jedná se o aritmetické operátory, operátory porovnání a logické operátory.

Tabulka 4.1 Aritmetické operátory.

Operátor	Operace	Popis
+	sčítání	Součet dvou číselných hodnot
-	odečítání	Rozdíl dvou číselných hodnot
*	násobení	Součin dvou číselných hodnot
/	dělení	Podíl dvou číselných hodnot
%	zbytek po dělení (modulo)	Zbytek po celočíselném dělení

Aritmetické operátory jsou k dispozici prakticky v každém programovacím jazyce a mají vždy stejný význam jako v matematice. Ani v PHP tomu není jinak. Priorita operátorů je také stejná jako v matematice. Násobení, dělení a zbytek po dělení mají vyšší prioritu než sčítání a odečítání. K úpravě priority operátorů je možné použít kulaté závorky.

Výpis 4.18 Ukázka použití aritmetických operátorů.

```
<?php
  $a = 10;
  $b = 5;
  $c = 2;
  $d = $a + 2; // $d = 12
  $e = $b % 2; // $e = 1
  $f = $a + $b * $c; // $f = 20
  $g = ($a + $b) * $c; // $g = 30
?>
```



Operátory porovnání slouží ke srovnání dvou hodnot (nejčastěji dvou proměnných nebo proměnné a nějaké konstanty).

Tabulka 4.2 Porovnávací operátory.

Operátor	Operace	Výsledek
==	rovnost	`true`, pokud jsou porovnávané hodnoty stejné
!=	nerovnost	`true`, pokud jsou porovnávané hodnoty různé
<>	nerovnost	`true`, pokud jsou porovnávané hodnoty různé
>	větší	`true`, pokud je první hodnota větší než druhá
<	menší	`true`, pokud je první hodnota menší než druhá
>=	větší než	`true`, pokud je první hodnota větší nebo rovna než druhá
<=	menší než	`true`, pokud je první hodnota menší nebo rovna než druhá
===	shodnost	`true`, pokud jsou porovnávané hodnoty stejného typu a jsou si rovny
!==	neshodnost	`true`, pokud jsou porovnávané hodnoty různé a jsou různého typu

Při použití porovnávacích operátorů mohou být porovnávány i výrazy (v následujícím výpisu je to přiřazení do proměnné `$f`.

Výpis 4.19 Ukázka použití porovnávacích operátorů.

```
<?php
  $a = 10;
  $b = 5;
  $c = 2;
  $d = $a > 9; // $d = 1 (true)
  $e = $b == $c; // $e = 0 (false)
  $f = ($a - $b) > ($c + 2); // $f = 1 (true)
```



```
  $g = $b >= $c; // $g = 1 (true)
?>
```

Další skupinou operátorů jsou logické operátory, které slouží ke kombinaci podmínek. Jedna podmínka je vytvořena použitím jednoho porovnávacího operátoru. Pokud potřebujeme testovat více věcí najednou, spojíme jednotlivé podmínky pomocí logických operátorů. V PHP se používají tři logické operátory. Prvním logickým operátorem je negace. Tento operátor je unární a výsledkem je opačná logická hodnota než byla vstupní hodnota. Dalším logickým operátorem je spojka „a“ (případně „a zároveň“), která představuje logický součin. Jedná se o binární operátor, který vrací `true`, pokud jsou všechny dílčí podmínky `true`. Pokud je více podmínek spojenou pouze spojkou „a“, stačí aby jedna z nich byla `false` a výsledkem celého výrazu je `false`. Druhým binárním logickým operátorem je spojka „nebo“, která vrací `true`, pokud alespoň jedna z dílčích podmínek byla `true`. Tento operátor představuje logický součet.

Tabulka 4.3 Logické operátory.

Operátor	Název	Popis
`!`	negace	z `true` udělá `false` a naopak
`&&`	a	`true`, pokud obě porovnávané části jsou `true`
`||`	nebo	`true`, pokud alespoň jedna porovnávaná část je `true`
`and`	a	`true`, pokud obě porovnávané části jsou `true`
`or`	nebo	`true`, pokud alespoň jedna porovnávaná část je `true`

Jak je vidět v tabulce, v PHP existují dva logické operátory pro logický součin a dva operátory pro logický součet. Pokud použijeme operátor `&&` nebo operátor `and`, výsledek bude stejný. Totéž platí pro operátory `||` a `or`. Rozdíl je ale v prioritě operátorů. Nejvyšší prioritu z logických operátorů má negace `!`. Následují operátory `&&`, `||`, `and` a `or`. Operátor `||` má tedy vyšší prioritu než `and`, i když operátor `and` představuje logický součin a operátor `||` logický součet.

Výpis 4.20 Ukázka použití logických operátorů.



```
<?php
  $a = 10;
  $b = 5;
  $c = 2;
  $d = ($a > 0) && ($b > 0) && ($c > 0); // $d = 1
   (true)
```

```
  $e = ($a > $b) && ($c > $b); // $e = 0 (false)
  $f = !$d; // $f = 0 (false)
  $g = ($b < 10) || ($c > 10); // $g = 1 (true)
  $h = ($b < $a) and ($b > 8) || ($c > 0); // $h = 1
   (true)
?>
```

Dalším operátorem v PHP je operátor pro spojování řetězců, který je reprezentován znakem `.` (tečka). S tímto operátorem jsme se již v předchozích kapitolách setkali.

Výpis 4.21 Ukázka použití operátoru pro spojení řetězců.

```
<?php
  $jmeno = "Rostislav";
  $prijmeni = "Miarka";
  $cele_jmeno = $jmeno . " " . $prijmeni;
  echo $cele_jmeno; // vypíše "Rostislav Miarka"
?>
```



4.4.3 Ternární operátor

Posledním operátorem, který zde uvedeme, je ternární operátor. Unární operátory pracují s jedním operandem, binární pracují se dvěma operandy a ternární operátor pracuje se třemi operandy. Ternární operátor je v PHP pouze jeden a označuje se pomocí znaků `?` `:` (otazník, dvojtečka). Pomocí tohoto operátoru vytvoříme konstrukci, která se skládá ze tří částí. První částí je logická podmínka, za kterou následuje znak `?` (otazník). Za otazníkem následuje hodnota, která je vrácena, pokud je podmínka vyhodnocena jako `true`. Na tomto místě může být uveden i příkaz. Další v pořadí je uvedena `:` (dvojtečka) a za ní následuje hodnota, která je vrácena, pokud byla podmínka vyhodnocena jako `false`. Ve své podstatě nahrazuje ternární operátor příkaz `if … else`, se kterým se seznámíme v následující kapitole.

Výpis 4.22 Ukázka použití ternárního operátoru.

```
<?php
  $a = 10;
  $a > 0 ? $b = "ano" : $b = "ne";
  $c = $a < 0 ? "ano" : "ne";
  echo $b; // vypíše "ano"
  echo $c; // vypíše "ne"
?>
```



4.4.4 Výrazy

Pojem výraz představuje použití několika různých operátorů stejného typu najednou. Použitím několika aritmetických operátorů současně vytvoříme aritmetický výraz. Pomocí logických operátorů vytváříme logické výrazy. Jejich součástí mohou být i aritmetické výrazy, ale naopak to neplatí.

Výpis 4.23 Ukázka výrazů.



```
<?php
  // aritmeticke vyrazy
  $a = 10;
  $b = 5;
  $c = -2;
  $a1 = 3 + $a * 2; // $a1 = 23
  $a2 = $a / $b - 20 + $c; // $a2 = -20
  $a3 = ($c * $c) + ($a * $b); // $a3 = 54

  // logicke vyrazy
  $b1 = ($a > $b) && ($a > $c); // $b1 = 1 (true)
  $b2 = $a + 5 > $b * 4 || $a / 2 < $b * 2; // $b2 =
   1 (true)
?>
```



4.4.5 Zkrácené přiřazení

Jazyk PHP umožňuje pro vybrané operátory používat zkrácené přiřazení, které je možné použít například v jazyce Java. V následující tabulce jsou uvedeny možností zkráceného přiřazení pro různé operátory.

Tabulka 4.4 Zkrácené přiřazení.

Zkrácené přiřazení	Nezkrácený zápis
`$a += $b;`	`$a = $a + $b;`
`$a -= $b;`	`$a = $a - $b;`
`$a *= $b;`	`$a = $a * $b;`
`$a /= $b;`	`$a = $a / $b;`
`$a %= $b;`	`$a = $a % $b;`
`$a .= $b;`	`$a = $a . $b;`

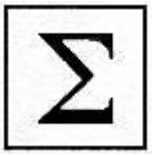


Kontrolní otázky:

1. K čemu slouží příkaz `echo`?
2. Jaké jednoduché datové typy můžeme použít v PHP?

3. Jaká pravidla musí splňovat identifikátor proměnné?
4. Jaké složitější datové typy můžeme použít v PHP?
5. Jaké operátory můžeme použít v PHP?

Shrnutí obsahu kapitoly



V této kapitole jste se seznámili s příkazem „echo“, který umožňuje vypsat text do okna prohlížeče. Dále jste se naučili práci s proměnnými, která je velmi důležitá pro tvorbu aplikací v jakémkoliv jazyce. Jazyk PHP umožňuje používat čtyři jednoduché datové typy – logickou hodnotu, celé číslo, desetinné číslo a řetězec znaků. Kromě toho umožňuje použití složitějších datových typů – polí a objektů. Pro tvorbu aplikací je možné někdy využít i tvorbu konstant. Jazyk PHP umožňuje používat unární a binární operátory a jeden ternární operátor.